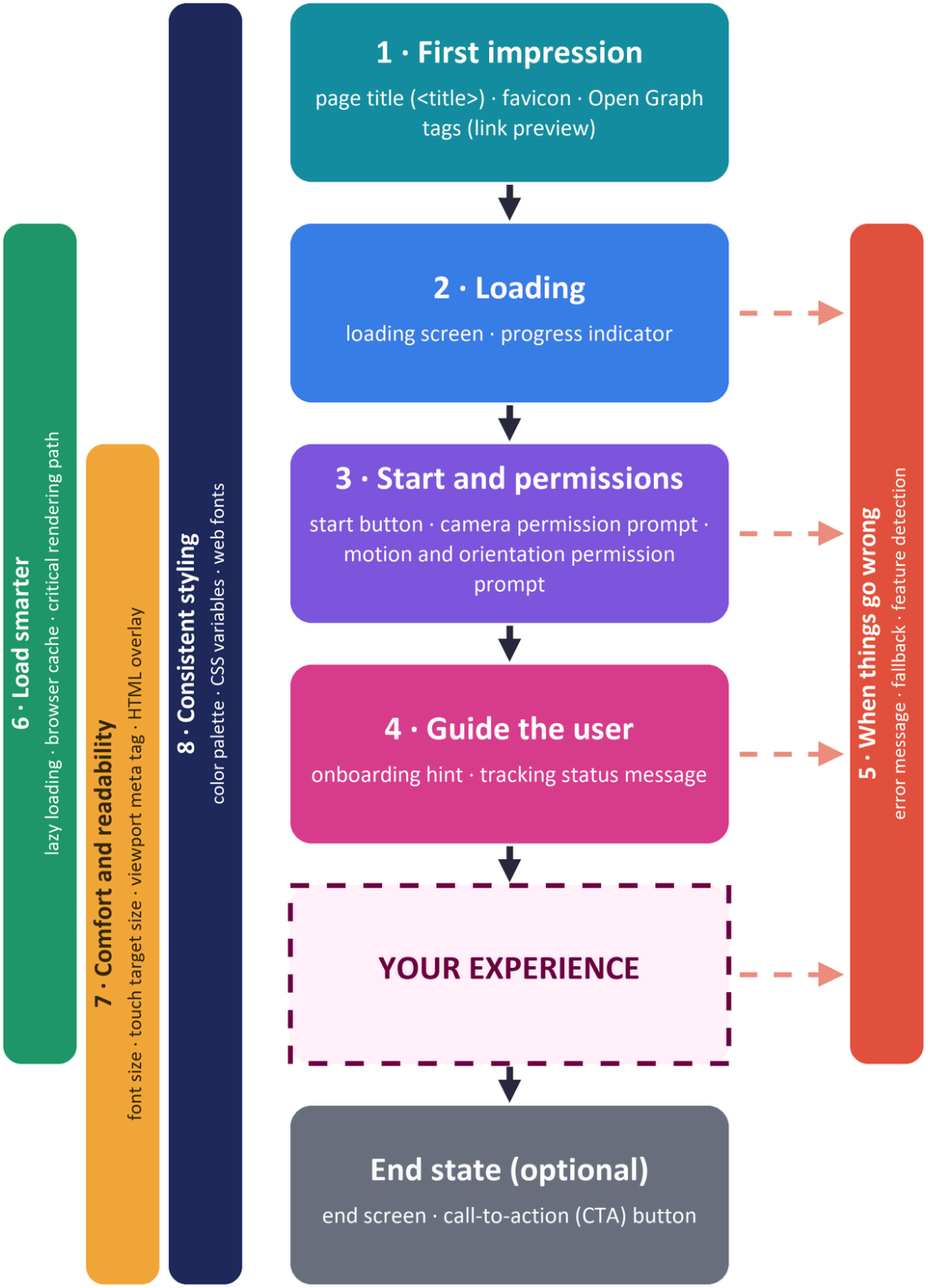


arrows = the user's path dashed arrows = if something fails bars = apply across several steps

Usability Flow In Detail

This document contains a summary of small things with big effects you can implement to take your experience from a tech preview to something that feels like a finished web app. Pick what you want to include in your project and add it to your features list.

1. First impression

- Page title (the <title> tag, shown in the browser tab)
- favicon (the small icon in the tab)
- link preview (the image and title that show when you send your link in a chat, controlled with Open Graph tags: og:title, og:image), so that your link looks like something.

2. Loading screen and progress

- Show something immediately: a logo, a title, a spinner, a progress bar or a short text ("Loading…").
- Show text, not only a spinner, so users know what they wait for ("Loading 3D model…").
- Remove it when the scene is ready. A-Frame fires a loaded event on <a-scene>. For assets in <a-assets>, a-asset-item fires progress (loaded and total bytes), loaded and error.

  Remember: if the loading takes longer than a few seconds, the user needs a sign that it is still working.

3. Start button and permissions

Camera and motion sensors need a permission, sound needs a user gesture.

- Optional: Start screen with one clear button ("Start AR"). The tap is the gesture the browser requires to play sound, sound that should play on load will be blocked otherwise. Without any of these, the camera prompt appears without a tap and the start screen is optional, but it is still the best place to explain the camera use.
- Handle "denied": if the user says no, the page should say what happened and how to fix it (browser settings), not stay black.
- Everything needs https and the camera works only on a secure connection.
- Impressum, privacy and contact: from the start screen on, keep a small link on every screen (start, scanning, end state).
  For a course project that is not public, you do not need an Impressum. As soon as a site is public and has a commercial or professional purpose it usually does (Switzerland: the Impressum rule is part of the UWG, and the privacy law is the revDSG). Camera images in WebAR are processed on the device, but if you use analytics, forms or external services, you need to say so.

4. Guide the user

- Onboarding hint: tell the user what to do when AR starts (In words and/or visual cues): "Point your camera at the postcard". Show it while no target is found, hide it when found.
- Tracking lost: tell the user when the target gets lost, and how to get it back.
- Tell users what they can do: if something is clickable, make it look clickable. A short "Tap the model" hint beats a hidden interaction nobody finds.
- Give feedback to every action: a small animation, color change or sound when the user taps something.

End state:
Optional. Closing the browser tab is the normal way out, so no exit button is needed. If there is a natural next step or call to action (CTA), offer it, for example a link to your website or a "play again" button.

5. When things go wrong: error messages and fallbacks

Things will fail: no camera, no WebXR support, no internet, model not found. The question is what the user sees then.

- Feature detection: before you start, check in code whether the browser and device support what your experience needs, for example camera access, WebXR, motion sensors or WebGL. Only offer the start button if it can work. Otherwise show a clear message or the fallback.
- Fallbacks: offer a simpler version when the full experience cannot run, from best to simplest. For example the same content in the normal browser window (touch or mouse to look around), or a still image or video with a short explanation. Whatever you choose, the user should never end up on a blank screen.
- Write human error messages: "We could not access your camera. Check the browser permission and reload." instead of a blank screen or a console error.
- Test the failure paths: switch off the camera permission, switch to airplane mode, break the file name of your model. See what the user sees.

Remember: a fallback does not need to look as good as the full experience. It only needs to not be a dead end.

6. Load smarter: deferred loading and caching

- Deferred (lazy) loading: do not load everything at the start if you don‘t have to. Load what the first view needs, load the rest later.
- Preload what is needed immediately: that is what <a-assets> is for in A-Frame.
- Caching: the browser keeps files it has already downloaded, so the second visit is much faster. That is free, but: Vite adds a hash to the file names of files it builds, so an update is picked up automatically. Files in your public/ folder keep their name by default, so users may still see the old version after you changed it. Rename those files or hash them as well if you update them in your live web app.
- Loading order (critical rendering path): the browser shows the page only as fast as it gets the files it needs, and the order matters. Late or slow styles and fonts cause flashes: FOUC (flash of unstyled content) is when the page shows briefly without its styles, FOUT (flash of unstyled text) when the text first shows in a default font and then jumps to your web font. Load your CSS in the head, keep font files small, or hide the page behind your loading screen until fonts and styles are ready.

7. Comfort and readability

- Text: large, high contrast, short. Test on the real device.
- Buttons: On phone, big enough for a finger (about 44 px minimum). Put 2D buttons and text in an HTML overlay instead of inside the 3D scene where possible: a normal HTML element on top of the canvas (in WebXR sessions this is the dom-overlay feature).
- Phone use: set the viewport meta tag, prevent accidental page scroll and zoom while using the experience, and consider that the screen can turn off while the user looks through the camera.

8. Consistent styling

- One color palette and one or two fonts for everything: page, loading screen, start button, hints, error messages and end state. Pick them once and write them down as CSS variables (--main-color, --font), so that you change them in one place.
- The same button style everywhere. A start button that looks different from the other buttons feels like two different apps.
- Restyle the defaults: for example, the built-in loading, scanning and error screens of MindAR are generic. Point them to your own elements so they match your look.
- Match the 3D scene: take the colors of your interface from your scene (or the other way around), so the page and the AR content feel like one piece.
- One language and one tone in all texts, and one icon style.

Remember: A custom web font has to be loaded before it shows. Until then the browser briefly shows a default font.

Checklist for your project

- [ ] Something intentional is visible right after opening the page
- [ ] Loading shows progress or at least text
- [ ] (Start button explains why the camera is needed)
- [ ] Hint on how to start (what to point at)
- [ ] Message when the tracking is lost
- [ ] Clear message when camera is denied or features are not supported
- [ ] Fallback so there is no dead end
- [ ] Big model loaded only when needed
- [ ] Text and buttons readable on the real device
- [ ] Title, icon, share preview
- [ ] One look everywhere: same colors, fonts and buttons
- [ ] Tested on real devices and browsers, in a bad network, with permissions denied